EXPERIMENT 09

Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

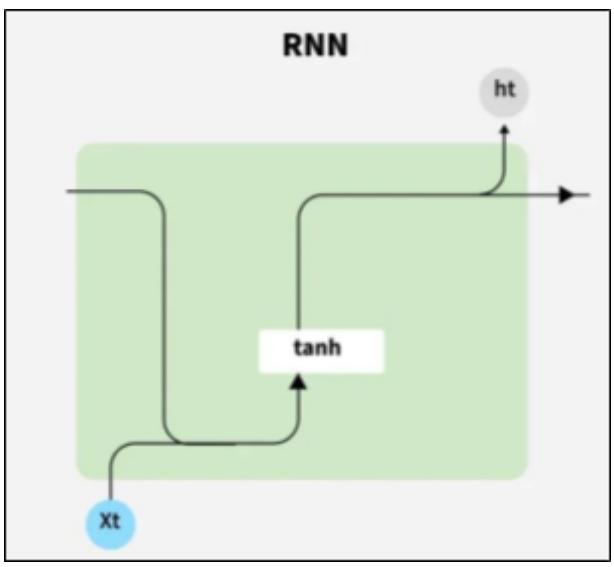
RNNs

RNNs are neural networks built specifically for handling sequential data. Unlike traditional feedforward networks they have loops that let them keep information from previous steps. This makes them useful for tasks where current outputs depend on earlier inputs like language modeling or predicting the next word.

The basic structure includes:

- Input Layer: Receives the sequence data.
- Hidden Layer: Processes input and maintains information from earlier time steps through recurrent connections.
- Output Layer: Generates predictions based on the current hidden state.

RNNs perform well on short sequences but struggle to capture long-range dependencies due to their limited memory.

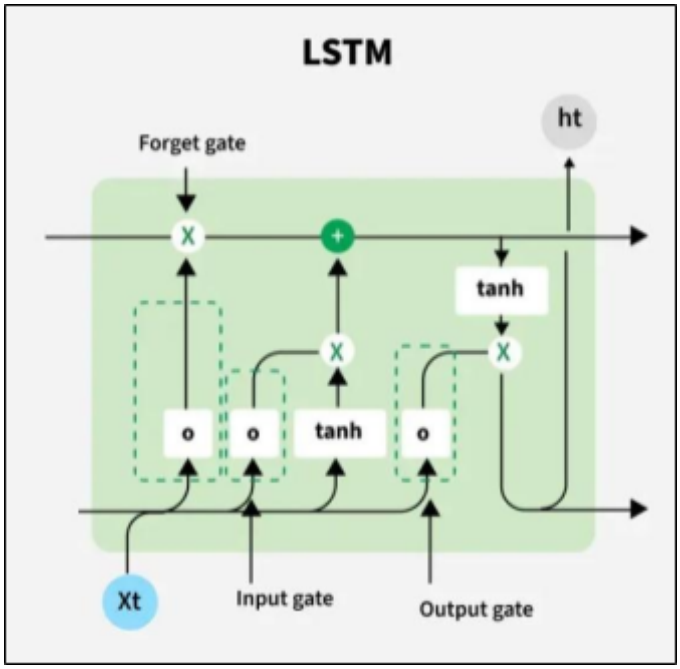


Long Short-Term Memory (LSTM) Networks

LSTM networks are an improved version of RNNs designed to solve the vanishing gradient problem. They use memory cells that keep information over longer periods.
LSTMs have special gates to control the flow of information:

1. Input Gate: Decides what new information to store.
2. Forget Gate: Choose what information to remove.
3. Output Gate: Decides what information to pass on.

This gating system allows LSTMs to remember and forget information selectively helps in making them effective at learning long-term dependencies.
They work well in tasks like sentiment analysis, speech recognition and language translation where understanding context over long sequences is important.



1.Dataset Source

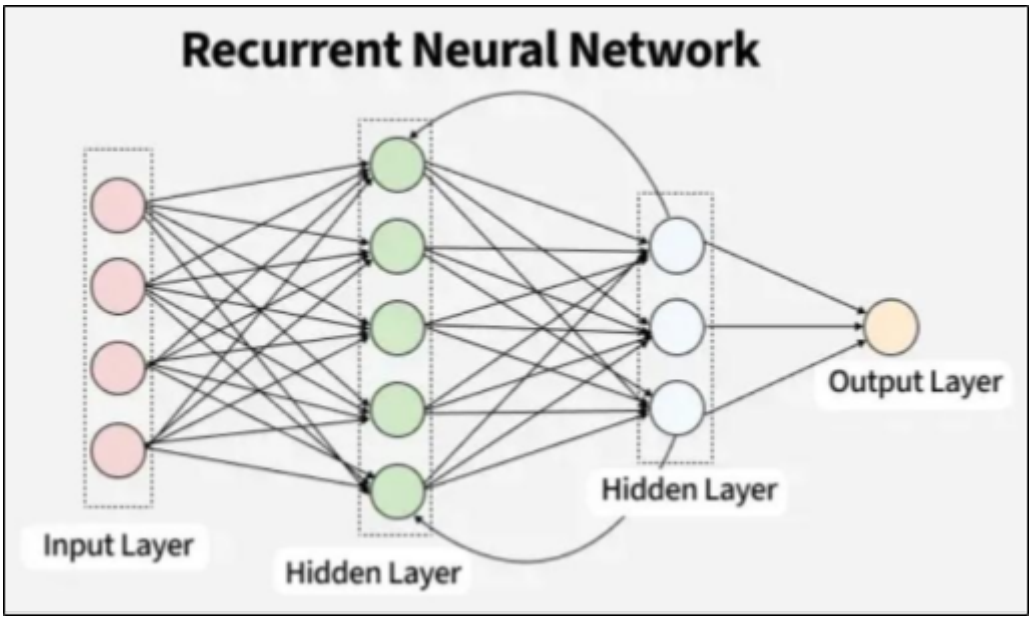
Dataset: Airline Passengers Dataset
Source:
https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.cs

v It contains monthly airline passenger counts.

2.Dataset Description

- **Type:** Time Series Data
- **Records:** ~144 months
- **Feature:**
  - Passengers → Number of passengers
- **Target Variable:**
  - Future passenger values

Since data depends on previous values → **Sequential data → RNN/LSTM used**



3.Mathematical Formulation

RNN Concept

RNN uses previous output as input:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Where:

- ht = hidden state
- xt = input
- W = weights

LSTM Concept (Improved RNN)

LSTM (Long Short-Term Memory) is a special type of Recurrent Neural Network (RNN) designed to learn from sequential data (like time series, text, speech) while solving the main problem of traditional RNNs — the vanishing gradient problem.

Why LSTM is Needed (Problem with RNN) ?

Normal RNNs:

- Struggle to remember **long-term dependencies**
- Gradients become very small during training (vanishing gradient)
- Forget earlier important information

LSTM solves vanishing gradient problem using gates:

1. Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t])$$

2. Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t])$$

3. Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t])$$

These gates control memory flow.

4. Algorithm Limitations

- RNN suffers from **vanishing gradient problem**
- LSTM is **computationally expensive**
- Requires **large sequential data**
- Training is **slow**
- Hard to interpret

5. RNN / LSTM Workflow

Step 1: Data Collection

First, we collect sequential data.

In this experiment:

- Airline passenger data (monthly values)

Key point:

- Data must be **time-dependent / sequential**

Example:
Passengers in Feb depend on Jan → sequence matters

Step 2: Data Visualization

Before applying the model, we visualize the data.

- Plot graph (Time vs Passengers)
- Identify trends, seasonality,

patterns Why is it important?

- Helps understand behavior of data
- Confirms it is suitable for time series modeling

Step 3: Data Preprocessing

Raw data cannot be directly used.

✔ Normalization

- Scale values between **0 and 1**
- Done using MinMaxScaler

Why?

- Neural networks perform better on scaled data

Step 4: Creating Sequences

This is the core concept of RNN/LSTM.

Instead of single input, we use **sequence of past values**

Example:

If time_step = 3

Input (X)	**Output (y)**
[100, 120, 130]	140
[120, 130, 140]	150

Model learns:
"Next value depends on previous values"

Step 5: Reshaping Data

LSTM expects 3D input:

(samples,time steps,features)(samples, time\ steps, features)(samples,time steps,features)

Example:

- 100 samples
- 5 time steps
- 1 feature

Shape = (100, 5, 1)

Step 6: Train-Test Split

Split data into:

- **Training Data (80%)** → model learns
- **Testing Data (20%)** → model is evaluated

Important for checking real performance

Step 7: Model Building (LSTM Architecture)

Now we create the model.

Components:

1. **LSTM Layer**
    - Stores memory
    - Learns patterns over time
2. **Dense Layer**
    - Produces final output

Flow:

Input → LSTM → Dense → Output

Step 8: Model Compilation

We define how model will learn:

- **Optimizer:** Adam
- **Loss Function:** MSE (Mean Squared

Error) Purpose:

- Optimize weights
- Reduce prediction error

Step 9: Model Training

The model is trained using training data.

- Multiple epochs (iterations)
- Batch processing
- Validation data used

During training:

- Loss decreases
- Accuracy improves

Step 10: Prediction

After training:

- Model predicts values for test data
- Output is in scaled form

So we apply **inverse scaling** to get actual values

Step 11: Performance Evaluation

We measure model performance using:

✔ MSE

Difference between actual and predicted values

✔ RMSE

Square root of MSE (easy to interpret)

Lower values = better model

Step 12: Visualization of Results

Plot graph:

- Actual values
- Predicted values

If both lines are close:
✔ Model is accurate

<u>Complete Workflow Summary</u>

Data Collection
↓
Visualization
↓
Preprocessing (Scaling)
↓
Sequence Creation
↓
Reshaping
↓
Train-Test Split
↓
Model Building (LSTM)
↓
Compilation
↓
Training
↓
Prediction
↓
Evaluation (MSE, RMSE)
↓
Visualization

6.Performance Analysis

Metrics used:

- **MSE (Mean Squared Error)**
- **RMSE**

Lower error = better prediction

7.Hyperparameter Tuning

Parameters tuned:

- Units (neurons)
- Epochs
- Batch size
- Learning rate

Helps improve accuracy and reduce loss

Conclusion

In this experiment, a Recurrent Neural Network (RNN) using Long Short-Term Memory (LSTM) was successfully implemented for time series prediction. The model was trained on sequential data to learn patterns and dependencies from past observations.

Through proper data preprocessing, sequence generation, and model training, the LSTM model was able to predict future values with good accuracy. Performance evaluation using metrics such as Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) showed that the model effectively captured trends in the dataset.